OLIcyber – Web 11
HTTP: CSRF Tokens

In this challenge, the flag is divided into four pieces, accessible through the resource `http://web-11.challs.olicyber.it/flag_piece`in basic to the parameter `index`. . To access it, you need to have Previously sent a POST login request to the resource `http://web-11.challs.olicyber.it/login`, with body JSON containing `"username": "admin"`and and `"password": "admin"`, and received in return a session cookie. In addition

This challenge requires us to submit CSRF tokens to a specific endpoint after login on another webpage

OLIcyber – Web 11
What are CSRF Tokens?

```
CSRF Tokens - Typical Actions

- Email Change    - Password Change
- Disabling MFA   - Security Question Change
```

CSRF (Cross-Site Request Forgery) tokens are a security mechanism that ensure that sensitive website actions, such as account email change or changing an account password, are done in the context of the correct user

OLIcyber – Web 11
What are CSRF Tokens?

```
1) Hacker sends link to victim
2) Victim clicks on link,
   which changes their password
3) Hacker logs into victim account
   using the changed password
```

If CSRF tokens are not used for sensitive actions, there's potential for abuse: e.g., tricking a user into clicking on a link which changes their account email, which can lead to account takeover

OLIcyber – Web 11
What are CSRF Tokens?

```
1) Hacker sends link to victim
2) Victim clicks on link,
   which changes their password
3) Hacker logs into victim account
   using the changed password
```

If CSRF tokens are not used for sensitive actions, there's potential for abuse: e.g., tricking a user into clicking on a link which changes their account email, which can lead to account takeover

OLIcyber – Web 11
What are CSRF Tokens?

Depending on the website, CSRF tokens can be:

- generated with the page where you can execute the sensitive actions
- given as a cookie when logged in
- sent to you as a JSON object

OLIcyber – Web 11
What are CSRF Tokens?

```
{
  "error": "Unauthorized",
  "message": "Bad csrf token"
* Connection #0 to host web-1
1.challs.olicyber.it:80 left
intact
}
```

If we try to perform a sensitive action on a website without using the correct CSRF token, the action fails

OLIcyber – Web 11
Curl Solution

```
└─$ curl -X POST -H "Content-Type: application/json" -d '{"username":
eb-11.challs.olicyber.it/login -c cookies.txt
Note: Unnecessary use of -X or --request, POST is already inferred.
* Host web-11.challs.olicyber.it:80 was resolved.
```

To solve this challenge with the Curl, we have to first make a request to the login endpoint with credentials in the form of a JSON object

OLIcyber – Web 11
Curl Solution

```
└─$ curl -v 'http://web-11.challs.olicyber.it/flag_piece?index=0&csrf=76961faf6c3a337d'
* Host web-11.challs.olicyber.it:80 was resolved.
* IPv6: (none)
* IPv4: 5.75.221.48
*   Trying 5.75.221.48:80 ...
* Established connection to web-11.challs.olicyber.it (5.75.221.48 port 80) from 10.0.2.
* using HTTP/1.x
```

Then we have to send a series of GET requests to the /flag_piece endpoint, specifying the index number and the CSRF token as URL parameters, updating the CSRF token value as we go

OLIcyber – Web 11
Python Solution

```
with requests.Session() as s:
  # login with credentials with POST
  login_res = s.post(url=login_URL, json=credentials)
```

To solve this challenge with Python, we need to use the Requests module to setup a Session object, so we can persist our session cookie across multiple requests

OLIcyber – Web 11
Python Solution

```python
# retrieve the CSRF token in the response
csrf_token = login_res.json()["csrf"]
```

```python
for i in range(4):
  res = s.get(url=flag_URL, params={"index": i, "csrf": csrf_token})
  print(f"Sending request number {i} to flag endpoint:")
  print(f"Received the following response:\n{res.text}")
  # update the flag parts and the CSRF token
  flag_parts.append(res.json()["flag_piece"])
  csrf_token = res.json()["csrf"]
```

After the login request, we can record the CSRF token in a variable, then make a new request to the /flag_piece endpoint with specified parameters, updating the CSRF token with each request and record each flag piece